



BENG INDIVIDUAL PROJECT

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Proving Program Equivalence In Dafny

Author:
Nathaniel Victor

Supervisor:
Dr Dragana Milovancevic
Prof. Sophia Drossopoulou

Second Marker:
Dr Mark Wheelhouse

January 20, 2026

Abstract

Two programs are equivalent if they produce the same output for any input. Proving program equivalence has applications across a variety of fields, however, these proofs are often difficult to write. This has led to the development of a variety of tools to help mitigate this [10, 9, 6, 3], but using them still requires significant mathematical expertise that the average software developer does not have. The aim of this project is to decrease the necessity for this as much as possible, focussing specifically on the Dafny program verifier. In this project, I will explore Dafny's capabilities and limitations in proving program equivalence. I will build a tool on top of Dafny to attempt to remove these limitations and allow the user to create equivalence proofs with as little user assistance as possible. Finally, I will evaluate this tool against the current state-of-the-art systems.



Contents

1	Introduction	4
2	Background	5
2.1	Dafny	5
2.2	SymDiff	5
2.3	Stainless	5
2.4	KLEE	5
3	Technical Audit	7
3.1	Necessary Datastructures	8
3.2	Incorrect Counterexamples	8
3.3	Equivalence Lemmas for Helper functions	9
3.4	Generating Auxiliary Lemmas	11
3.5	Explicitly enforcing induction to prove lemmas	12
3.6	Providing Variants to prove termination	13
3.7	Equivalence of Imperative Methods	15
3.7.1	Proving Equivalence of Imperative Methods using Product Programs	15
3.7.2	Equivalence of Methods and Functions	17
3.8	Equivalence of Higher-Order Functions	18
4	Plans	19
4.1	Project Plan	19
4.2	Evaluation Plan	20
5	Ethical, Legal, Professional and Societal Issues	22



Listings

(not usually needed)

3.1	<code>isSortedR</code> function with <code>loop</code> helper function	7
3.2	<code>isSortedB</code> Function	7
3.3	Equivalence Lemma for <code>isSortedR</code> and <code>isSortedB</code>	7
3.4	List Data Structure	8
3.5	<code>isSortedA</code> Function	8
3.6	Equivalence Lemma for <code>isSortedA</code> and <code>isSortedR</code>	9
3.7	Equivalence Lemma with Helper for <code>isSortedA</code> and <code>isSortedR</code>	9
3.8	<code>isSortedC</code> Function	9
3.9	Equivalence Lemma with Helper for <code>isSortedC</code> and <code>isSortedR</code>	10
3.10	<code>validAdjListM</code> and <code>validAdjList</code> Functions	10
3.11	Equivalence Lemma for <code>validListM</code> and <code>validList</code>	10
3.12	Equivalence Lemma for <code>validAdjListM</code> and <code>validAdjList</code>	10
3.13	<code>limit3_1</code> and <code>limit3_2</code> Functions	11
3.14	Auxiliary and Equivalence Lemmas for the <code>limit3_1</code> and <code>limit3_2</code> Functions	11
3.15	<code>m1</code> and <code>m2</code> Functions	11
3.16	Auxiliary Lemma for <code>m2</code>	12
3.17	Equivalence Lemma for <code>m1</code> and <code>m2</code>	12
3.18	<code>leVraiSculpteurDeNuage</code> and <code>leVraiSculpteurDeNuageM</code> Functions	12
3.19	Equivalence Lemma for <code>mixmashM</code> and <code>mixmash</code>	12
3.20	Equivalence Lemma for <code>leVraiSculpteurDeNuageM</code> and <code>leVraiSculpteurDeNuage</code>	12
3.21	<code>maxR</code> Function	13
3.22	<code>maxC</code> Function	13
3.23	Equivalence Lemma for <code>maxC</code> and <code>maxR</code>	13
3.24	<code>sub</code> and <code>mySub</code> Functions	14
3.25	Equivalence Lemma for <code>sub</code> and <code>mySub</code>	14
3.26	<code>solution_2</code> and <code>uniq2</code> Functions	14
3.27	Equivalence Lemma for <code>drop_2</code> and <code>drop</code>	14
3.28	<code>lem2</code> Helper Lemma	14
3.29	Equivalence Lemma for <code>solution_2</code> and <code>uniq2</code>	15
3.30	<code>isSortedBI</code> Method	15
3.31	<code>isSortedRI</code> Method with <code>loopI</code> Helper Method	16
3.32	Product Program for <code>isSortedBI</code> and <code>isSortedRI</code> using <code>loopI</code>	16
3.33	<code>isSortedBI</code> Method with simplified specifications	17
3.34	<code>andThen1</code> and <code>andThen2</code> Functions	18
3.35	Equivalence Lemma for <code>andThen1</code> and <code>andThen2</code>	18
3.36	<code>flip1</code> and <code>flip2</code> Functions	18
3.37	Equivalence Lemma for <code>flip1</code> and <code>flip2</code> with specified arguments	18
3.38	Invalid Equivalence Lemma for <code>flip1</code> and <code>flip2</code>	18

Chapter 1

Introduction

Proving Program Equivalence is the act of verifying that, for any input, two programs always give the same output. If this can be proven, then these programs are described as equivalent; one of these programs can be substituted for the other without affecting the behaviour of the overall system. This has applications in a variety of fields. For example, compiler developers can use this to ensure that optimisations always preserve the intended behaviour of the original program. Software developers in general can use this to formally verify that their code is bug-free, removing the necessity for exhaustive testing, which is often infeasible for large projects.

Therefore, Program Equivalence is a very exciting area of Computer Science. However, it is a very difficult task, especially since the problem is undecidable in general. To mitigate this problem, a variety of tools are being developed to help prove equivalence automatically [10, 9, 6, 3]. These tools can make proving equivalence significantly easier for developers, especially when working with large codebases. They can also generate counterexamples when equivalence cannot be proven, making debugging much easier.

The main drawback of these tools is that, in general, they still require a significant level of user assistance to complete proofs, often involving writing complicated specifications, loop invariants or auxiliary lemmas. The average software developer will not have the necessary expertise to provide these, thus, these tools will be inaccessible to them. Therefore, the general aim of this project is to learn from the current state-of-the-art tools, and use this to build a new tool that decreases the level of user assistance required as much as possible, thus allowing the average software developer to better reap the benefits of using program equivalence tools in their work.

This project will begin with a review of the current tools for proving program equivalence in Chapter 2. After this, this project will focus solely on the Dafny program verifier. I will explore Dafny's capabilities and limitations in proving program equivalence in Chapter 3; this will clearly demonstrate that there are many cases where Dafny requires user assistance to complete its proofs. Thus, I will build a tool on top of the Dafny verifier that will allow the user to prove the equivalence of two Dafny functions with as little user assistance as possible. Finally, I will evaluate this tool against the current state-of-the-art systems as described in Chapter 4.

Objectives: ? (later this will become Contributions)

running example program
(+ spec?)

Chapter 2

Background

this section seems a bit short
should formally define program
equivalence
at the very least

2.1 Dafny

give an example
program/specification

Dafny is a popular program verifier, under active development by Amazon [10]. It is primarily a programming language, containing many of the classic language features and constructs we expect from a modern programming language. However, in addition to this, it includes specification constructs, such as preconditions, postconditions and loop invariants. This allows Dafny's verifier to automatically prove theorems and that programs match their specifications. Dafny can also be compiled into a variety of other languages, including C#, Java and Python, allowing verified programs to be integrated easily into existing codebases.

say something about how Dafny only compiles verified programs?

Dafny's program verification heavily relies on Boogie [2], another state-of-the-art program verifier. The verification process involves translating a Dafny program into a Boogie program; this is then translated into a series of verification conditions, which are passed to the Z3 SMT solver [4].

2.2 SymDiff

how does that work? example

SymDiff is a differential program verifier [9]: it works by specifying the differences between multiple programs and verifying properties about them.

SymDiff can also be used in Approximate Computing, as described in Lahiri et al. [7]. This is where you trade off the accuracy of a program (in an acceptable manner) for improvements in performance or energy consumption. Lahiri et al. [7] focussed on formally proving control flow equivalence under given differences, by constructing a product program, using the concept of a mutual summary, and passing it to the Boogie program verifier.

need to define these terms

2.3 Stainless

examples?

Stainless is a verification framework that verifies that Scala programs obey their specifications [6]; when these specifications are violated, Stainless is able to provide counterexamples. Stainless performs these verifications by passing each part of the program through a series of transformations, until it can be used by the Inox solver [5]. However, an important restriction of Stainless is that it only supports a subset of Scala's features.

which subset? what is excluded?

One part of Stainless's functionality that is especially relevant to this work is its equivalence checker [6, 11]. Its ability to automatically generate equivalence lemmas for helper functions and to match many functions at once will make it a great inspiration for the rest of this project.

meaning?

2.4 KLEE

example?

KLEE is a symbolic execution engine that automatically generates high coverage test sets [3, 12]. KLEE achieves this generation by simulating program execution with symbolic inputs, to explore every possible execution path a program can take. For each path, KLEE maintains the set of conditions needed for that path to have been taken, thus, if the path ends in an error, KLEE can use those conditions to generate a single test case that will create the error. Therefore, KLEE is a powerful tool, especially when working

why?

on program equivalence, however, one of its main limitations is that the number of possible paths to keep track of can grow exponentially large when it is working with programs containing many loops and branches.

this is all very light touch
need to expand on any technical terminology that you
will use, or build on, later.

Chapter 3

isn't this just more Background content?

Technical Audit

Dafny is a powerful tool for proving program equivalence. It is often able to prove that two given functions are equivalent fully automatically. Consider the following example, converted from the original Scala provided by Milovančević and Kunčak [11]. These two examples attempt to check whether a list is sorted. The first is as follows, involving a helper function `loop` and the main function `isSortedR`:

```
function loop(p: int, l: seq<int>): bool
  decreases l
{
  if |l| == 0 then true
  else p <= l[0] && loop(l[0], l[1..])
}

function isSortedR(l: seq<int>): bool
{
  if |l| == 0 then true
  else loop(l[0], l[1..])
}
```

*tail recursive
vs. forward recursive
(less) costlier*

Listing 3.1: `isSortedR` function with `loop` helper function

The second is as follows:

```
function isSortedB(l: seq<int>): bool
  decreases |l|
{
  if |l| == 0 then true
  else if |l| == 1 then isSortedB([])
  else l[0] <= l[1] && isSortedB(l[1..])
}
```

*why not compare recursive
function to iterative one?
these two seem 'obviously'
the same, just some
indirection involved*

Listing 3.2: `isSortedB` Function

To prove the equivalence of these two functions, we can define the following lemma:

```
lemma isSortedEquivalence(l: seq<int>)
  ensures isSortedR(l) == isSortedB(l)
{}  
?
```

Listing 3.3: Equivalence Lemma for `isSortedR` and `isSortedB`

*nice to
see an
example*

Thus, Dafny is capable of proving that these functions are equivalent. As described in Section 2.1, this is achieved by translating the Dafny program into Boogie. For this example, this results in a file almost 3500 lines long. It begins with a series of type definitions and axioms, followed by the translation of each Dafny function, and ends with the translation of the equivalence lemma, taking only around 50 lines of code. Therefore, the Dafny programming language is highly expressive, allowing users to define functions in a concise manner. Meanwhile, the Dafny program verifier is capable of proving the equivalence of some functions without much effort on the user's part.

*but do explain where
this means in the prove
(implicit Vt?)*

However, there are many cases where the user needs to provide assistance to Dafny in order to prove equivalence. In the rest of this section, I will provide a series of examples to demonstrate where Dafny's limitations are in this area, and how the user can provide assistance to allow Dafny to overcome these limitations and prove equivalence. These limitations include a series of problems when generating and proving auxiliary lemmas, problems with proving termination of the equivalence lemmas in the presence

isn't the decreases l above additional user assistance

of termination proofs for the individual functions, and problems with proving the equivalence of Dafny methods with imperative features, such as loops.

The majority of the following examples have been taken from the program equivalence tests in the GitHub repository for the Stainless Scala verification tool [14]. I converted all of their tests into Dafny functions, and proved their equivalence. Many of these tests consisted of more than 2 functions, each in their own file. For these cases, I concatenated all of these functions into a single Dafny file to form a single test, and proved the equivalence of each function with a single function, thus proving, through the transitivity of equivalence, that all of these functions were pairwise equivalent. This resulted in a total of 62 tests.

These tests cover a great variety of types of programs. The number of lines of code in each test ranges from around 10 to over 200. They cover a variety of inbuilt and custom data structures, including sequences, lists and the Option datatype. Some tests also cover concepts such as mutual recursion, generic types and higher-order functions.

Using these tests, I have attempted to characterise the cases where Dafny required user assistance. In this way, I have identified 7 different types of limitations. Table 3.1 summarises the number of tests that demonstrate each type of limitation. Many examples demonstrate multiple types of limitations, therefore, these are counted multiple times. This table also includes imperative equivalence tests that I wrote myself. All of the tests from the Stainless repository can be proven equivalent by Stainless fully automatically, except for the single test involving higher-order functions.

consider
incorrect
prog-corr
too.

Type of Limitation	Number of Examples	Source
None (Fully Automatic)	19	[14]
Incorrect Counterexamples	1	[14]
Equivalence Lemmas for Helper functions	17	[14]
Generating Auxiliary Lemmas	3	[14]
Explicitly enforcing induction to prove lemmas	4	[14]
Providing Variants to prove termination	17	[14]
Equivalence of Imperative Methods	7	New
Equivalence of Higher-Order Functions	1	[14]

Table 3.1: Summary of the number of examples that demonstrate each type of limitation

The following sections describe each type of limitation that I have identified. Each section contains at least one example to clearly demonstrate the limitation being discussed. I have tried to pick the simplest, minimal examples that I have access to, to remove any unnecessary difficulty.

3.1 Necessary Datastructures

I will require the following definition of a list data structure:

```
datatype List<T> = Nil | Cons(head: T, tail: List<T>)
```

Listing 3.4: List Data Structure

are there any limitations
that only arise with
more complex data structures?

3.2 Incorrect Counterexamples

An example is as follows:

```
function leq(cur: int, next: int): bool
{
  cur < next
}

function iter(l: List<int>): bool
{
  match l
  case Nil => true
  case Cons(x, Nil) => true
  case Cons(x, Cons(y, ys)) => leq(x, y) && iter(Cons(y, ys))
}
```

```

function isSortedA(l: List<int>): bool
{
  match l
  case Nil => true
  case Cons(_, Nil) => true
  case Cons(x, Cons(y, ys)) => x <= y && iter(Cons(y, ys))
}

```

Listing 3.5: isSortedA Function

This solution is non-equivalent to the reference solution detailed in Listing 3.1, as it uses a strictly less-than comparison in the `leq` function, when it should use a less-than-or-equal-to comparison. Therefore, Dafny is unable to prove the following equivalence lemma:

```

lemma isSortedEquivalence(l: List<int>)
  ensures isSortedA(l) == isSortedR(l)
{}

```

Listing 3.6: Equivalence Lemma for isSortedA and isSortedR

However, it incorrectly provides `Cons(-38, Cons(7681, Nil))` as a counterexample. I extended the above proof to include and use a helper lemma, as shown below:

```

lemma isSortedEquivalenceHelper(x: int, xs: List<int>)
  decreases xs
  ensures iter(Cons(x, xs)) == loop(x, xs)
{}

lemma isSortedEquivalence(l: List<int>)
  ensures isSortedA(l) == isSortedR(l)
{
  match l
  case Nil => {}
  case Cons(x, Nil) => {}
  case Cons(x, Cons(y, ys)) => isSortedEquivalenceHelper(y, ys);
}

```

Listing 3.7: Equivalence Lemma with Helper for isSortedA and isSortedR

Now, Dafny is able to provide a correct counterexample of `x = 8855` and `xs = Cons(8855, Nil)` for the helper lemma. However, this is not the simplest counterexample possible, as `x = 0` and `xs = Cons(0, Nil)` would be sufficient.

As asserted by Putruele et al. [13], since there are multiple translations steps between the Dafny source code and the verification conditions sent to the Z3 SMT solver, the counterexamples produced by it can be difficult to read and understand. They also note these counterexamples may sometimes be incorrect, due to the Z3 solver being incomplete.

3.3 Equivalence Lemmas for Helper functions

The example is as follows:

```

function chk(l: List<int>, p: int, a: bool): bool
{
  match l
  case Nil => a
  case Cons(x, xs) => x >= p && chk(xs, x, a)
}

function isSortedC(l: List<int>): bool
{
  match l
  case Nil => true
  case Cons(x, xs) => chk(Cons(x, xs), x, true)
}

```

Listing 3.8: isSortedC Function

This example is equivalent to the reference solution detailed in Listing 3.1, however, Dafny requires a helper lemma to prove this:

```

lemma isSortedEquivalenceHelper(x: int, xs: List<int>)
  decreases xs
  ensures chk(xs, x, true) == loop(x, xs)
{}

lemma isSortedEquivalence(l: List<int>)
  ensures isSortedC(l) == isSortedR(l)
{
  match l
  case Nil => {}
  case Cons(x, xs) => isSortedEquivalenceHelper(x, xs);
}

```

Therefore, for Dafny to verify the equivalence of the main functions `isSortedC` and `isSortedR`, it must also verify the equivalence of its helper functions `chk` and `loop`. This means that the user must provide additional helper lemmas to prove the equivalence of these helper functions.

Additionally, the user must explicitly call these helper lemmas, with the correct arguments, within the main equivalence lemma (as demonstrated above). Sometimes, these helper lemmas can only be called when specific conditions are met. To demonstrate this second point, I will provide two implementations of a function to determine whether a given adjacency list is valid. They are as follows: *meaning?*

I don't
follow
this
example

The definitions of `validListM` and `validList` are irrelevant for this example, so I will omit them. All that is required is that they are equivalent:

it's hard to tell what is going on without them..

Dafny can now use this lemma to prove the equivalence of `validAdjListM` and `validAdjList`, but only if it is used when a specific condition is met:

highlight & the
solved condition

10

Dafny must verify that `pos - 1` is a valid index for `adjList` before the lemma `equivalenceValidList` can be called. Therefore, the lemma can only be called under the condition that `pos > 0`.

Therefore, to prove the equivalence of two functions that both use helper functions, the user must first prove the equivalence of the helper functions, and then explicitly call the corresponding lemmas within the main equivalence lemma, ensuring that all required conditions are met at the call site. ✓

3.4 Generating Auxiliary Lemmas

Sometimes, to prove the equivalence of two functions, Dafny requires helper lemmas that will be difficult to generate automatically, since they require an understanding of what the functions being compared do. For example, consider the following two equivalent functions `limit3_1` and `limit3_2`:

```
function limit3_1(n: int): int {
  if (n <= 1) then n
  else n + limit3_1(n-1)
}

function limit3_2(n: int): int {
  if (n <= 1) then n
  else
    var r := limit3_2(n-1);
    if (r >= 0) then n + r
    else r
}
```

maybe briefly state what this program does/outputs? or give a precondition

Listing 3.13: `limit3_1` and `limit3_2` Functions

The main difference between these two functions is in the `else` branch of the `limit3_2` function, where it will either output `n + r` or `r`, depending on whether `r >= 0`. However, we can prove that `r` is always strictly positive in this case, thus `n + r` will always be returned. Dafny requires some help to realise this, and to prove equivalence: ✓

```
lemma equivalenceLimit3_Helper(n: int)
  requires n >= 1
  ensures limit3_2(n) > 0
{}

lemma equivalenceLimit3(n: int)
  ensures (limit3_1(n) == limit3_2(n))
{
  if n >= 2 { equivalenceLimit3_Helper(n); }
}
```

again, would be good to briefly explain what this is telling Dafny to do

Listing 3.14: Auxiliary and Equivalence Lemmas for the `limit3_1` and `limit3_2` Functions

Therefore, Dafny is able to prove the equivalence of these two functions, but only after the user provides a helper lemma that would be difficult for Dafny to automatically generate. Here is another, more complicated example. Consider the following two equivalent functions `m1` and `m2`:

```
function m1(n: int, flag: bool): int {
  if (n < 1) then 0
  else if (n == 1) then 1
  else m1(n - 1, !flag) + m1(n - 2, !flag)
}

function m2(n: int, mode: bool): int {
  if (n < 1) then 0
  else if (n == 1 || n == 2) then 1
  else
    var results := if (mode) then
      m2(n-2, !mode) + m2(n-2, !mode) + m2(n-3, !mode)
    else
      m2(n-1, !mode) + m2(n-2, !mode);
    results
}
```

which do...?

Listing 3.15: `m1` and `m2` Functions

The main difference between these two functions is in the `else` branch of the `m2` function, where it will output one of two different expressions depending on the value of `mode`. Because of this, Dafny requires an auxiliary lemma to prove equivalence:


```
lemma m2_equivalence_fib(n: int)
  requires n >= 2
  ensures m2(n, true) == m2(n - 1, false) + m2(n - 2, false)
{}
```

what info does this capture?

Listing 3.16: Auxiliary Lemma for m2

Using this, Dafny can now prove the equivalence of m1 and m2:

```
lemma equivalence(n: int, flag: bool)
  ensures m1(n, flag) == m2(n, flag)
{
  if n >= 2 { m2_equivalence_fib(n); }
}
```

Listing 3.17: Equivalence Lemma for m1 and m2

Thus, this is another example of where Dafny can prove the equivalence of two functions, but only after the user provides helper lemmas that would be difficult for Dafny to automatically generate.

3.5 Explicitly enforcing induction to prove lemmas

Dafny often proves lemmas through induction. However, sometimes, it requires the user to explicitly state how to perform this induction by calling the lemma the user is trying to prove recursively with ‘smaller’ arguments. For example, here are the two equivalent functions `leVraiSculpteurDeNuage` and `leVraiSculpteurDeNuageM`:

```
function leVraiSculpteurDeNuageM<A, B, C>(
  a: A, b: B, c: C, fuel: int, i1: int,
  i2: int, i3: int, a2b: A -> B, b2c: B -> C, c2a: C -> A
): (int, int, int)
  requires (fuel >= 0)
  decreases(fuel) {
    if (fuel == 0) then (i1, i2, i3)
    else
      var (ii1, ii2, ii3) := mixmashM(i1, i2, i3);
      leVraiSculpteurDeNuageM(
        c2a(c), a2b(a), b2c(b), fuel - 1, ii1, ii2, ii3, a2b, b2c, c2a
      )
  }

function leVraiSculpteurDeNuage<A, B, C>(
  b2c: B -> C, b: B, c: C, fuel: int, a2b: A -> B,
  i2: int, a: A, i3: int, i1: int, c2a: C -> A): (int, int, int)
  requires (fuel >= 0)
  decreases(fuel) {
    if (fuel == 0) then (i1, i2, i3)
    else
      var (ii1, ii2, ii3) := mixmash(i2, i3, i1);
      leVraiSculpteurDeNuage(
        b2c, a2b(a), b2c(b), fuel - 1, a2b, ii2, c2a(c), ii3, ii1, c2a
      )
  }
}
```

*This is pretty confusing
what is being achieved/calculated here?*

Listing 3.18: `leVraiSculpteurDeNuage` and `leVraiSculpteurDeNuageM` Functions

The definitions of `mixmashM` and `mixmash` are irrelevant for this example, so I will omit them. All that is required is that they are equivalent:

```
lemma equivalenceMixmash(i1: int, i2: int, i3: int)
  ensures (mixmash(i2, i3, i1) == mixmashM(i1, i2, i3))
  decreases (if (i2 <= 0) then -i2 else i2)
{}
```

Listing 3.19: Equivalence Lemma for `mixmashM` and `mixmash`

Using this lemma, Dafny can now prove the equivalence of the two functions, but only if the user explicitly calls the same lemma recursively:

```
lemma equivalenceLeVraiSculpteurDeNuage<A, B, C>(
  b2c: B -> C, b: B, c: C, fuel: int, a2b: A -> B,
  i2: int, a: A, i3: int, i1: int, c2a: C -> A
)
```

```

requires (fuel >= 0)
ensures (leVraiSculpteurDeNuage(b2c, b, c, fuel, a2b, i2, a, i3, i1, c2a) ==
        leVraiSculpteurDeNuage(a, b, c, fuel, i1, i2, i3, a2b, b2c, c2a))
{
  if (fuel > 0) {
    equivalenceMixmash(i1, i2, i3);
    var (ii1, ii2, ii3) := mixmash(i2, i3, i1);
    equivalenceLeVraiSculpteurDeNuage(
      b2c, a2b(a), b2c(b), fuel - 1, a2b, ii2, c2a(c), ii3, ii1, c2a
    );
  }
}

```

seen this style before?
 oh, so it's just reordering all of the params?
 there's probably a better example

Listing 3.20: Equivalence Lemma for `leVraiSculpteurDeNuageM` and `leVraiSculpteurDeNuage`

Therefore, in many cases Dafny is able to infer the correct induction method automatically, however, it sometimes requires the user to explicitly provide it themselves. *repetition*

3.6 Providing Variants to prove termination

Here, I provide two implementations of a function to find the maximum element in a sequence of integers. The first is as follows:

```

function maxR(l: seq<int>): int
{
  if |l| == 0 then -1
  else if |l| == 1 then l[0]
  else
    var m := maxR(l[1..]);
    if l[0] > m then l[0] else m
}

```

Listing 3.21: `maxR` Function

The second is as follows:

```

function maxC(l: seq<int>): int
  decreases |l|
{
  if |l| == 0 then -1
  else if |l| == 1 then l[0]
  else
    var a := l[0];
    var b := l[1];
    if a > b then
      maxC([l[0]] + l[2..])
    else
      maxC(l[1..])
}

```

can this be written a little more neatly?
 (i.e. without var a and b)

Listing 3.22: `maxC` Function

The main difficulty with proving the equivalence of these two functions is that they approach the problem in opposite ways: the first function processes the list from the end to the start, while the second processes it from the start to the end. However, despite this, Dafny is able to prove equivalence with almost no help from the user:

```

lemma maxEquivalence(l: seq<int>)
  ensures maxC(l) == maxR(l)
  decreases |l|
{}

```

Listing 3.23: Equivalence Lemma for `maxC` and `maxR`

Dafny requires a variant to prove termination, as it proves the equivalence lemma through induction. However, in this case, it is unable to infer this variant by itself so the user must provide it instead. This problem can be solved easily in some cases, including this one, by copying the variant from one of the functions being compared. ✓

This solution is especially apparent if both functions have the same variant. Here I provide two implementations of a function to subtract one integer from another. The two functions are as follows:

nice, more compelling example

```

function sub(x: int, y: int): int
  decreases(if (x <= 0) then -x else x) {
    if (x == 0) then -y
    else if (x > 0) then sub(x - 1, y - 1)
    else sub(x + 1, y + 1)
  }

// Computes y - x instead of x - y
function mySub(x: int, y: int): int
  decreases(if (y <= 0) then -y else y) {
    if (y == 0) then -x
    else if (y > 0) then mySub(x - 1, y - 1)
    else mySub(x + 1, y + 1)
  }

```

Listing 3.24: sub and mySub Functions

These two functions use the same variant to prove termination, and are equivalent if the arguments are matched correctly. Therefore, Dafny is able to prove their equivalence as follows, but only if the user explicitly copies the variant from one of the two functions:

```

lemma equivalenceSub(x: int, y: int)
  decreases(if (x <= 0) then -x else x)
  ensures (sub(x, y) == mySub(y, x))
{}

```

Listing 3.25: Equivalence Lemma for sub and mySub

In addition, if at least one of the functions requires a lemma or an assertion to prove termination, this must be copied into the equivalence lemma as well. For example, here are two implementations of a function to reduce a list down to its unique elements:

```

function solution_2(lst: List<int>): List<int>
  decreases(length(lst)) {
    match lst {
      case Nil => Nil
      case Cons(hd, tl) =>
        lemM(hd, tl);
        Cons(hd, solution_2(drop_2(tl, hd)))
    }
  }

function uniq2(lst: List<int>): List<int>
  decreases(length(lst)) {
    match lst {
      case Nil => Nil
      case Cons(hd, tl) =>
        lem2(hd, tl);
        Cons(hd, uniq2(drop(hd, tl)))
    }
  }

```

Listing 3.26: solution_2 and uniq2 Functions

The definitions of `drop_2` and `drop` are irrelevant for this example, so I will omit them. All that is required is that they are equivalent, when the arguments are matched correctly:

```

lemma equivalenceDrop(a: int, lst: List<int>)
  ensures (drop(a, lst) == drop_2(lst, a))
{}

```

Listing 3.27: Equivalence Lemma for drop_2 and drop

`solution_2` and `uniq2` also use helper lemmas `lemM` and `lem2` respectively to prove termination. These lemmas are equivalent, so I will only provide `lem2`:

```

lemma lem2(a: int, lst: List<int>)
  ensures length(drop(a, lst)) <= length(lst)
{}

```

Listing 3.28: lem2 Helper Lemma

Using `equivalenceDrop`, Dafny is able to prove the equivalence of `solution_2` and `uniq2`, but only if the user explicitly calls `lem2`:

```

lemma equivalence_solution2_uniq2(lst: List<int>)
  ensures (solution_2(lst) == uniq2(lst))
  decreases (length(lst))
{
  match lst
  case Nil => {}
  case Cons(hd, tl) =>
    lem2(hd, tl);
    equivalenceDrop(hd, tl);
}

```

Listing 3.29: Equivalence Lemma for `solution_2` and `uniq2`

Thus, if lemmas or assertions are needed to prove termination of at least one of the functions being compared, the user must explicitly copy them into the equivalence lemma as well. *repetition*

3.7 Equivalence of Imperative Methods

Recursive functions can be converted into their iterative versions. For example, the recursive implementation of `isSortedB`, described in Listing 3.2, can be converted into the following imperative method:

```

method isSortedBI(l: seq<int>) returns (res: bool)
  ensures res <==> (forall i :: 0 <= i < |l| - 1 ==> l[i] <= l[i+1])
{
  var currentL := l;

  while true
    decreases |currentL|
    invariant currentL == l[|l| - |currentL| .. ]
    invariant forall i :: 0 <= i < |l| - |currentL| ==>
      (i < |l| - 1 ==> l[i] <= l[i+1])
  {
    if |currentL| == 0 {
      return true;
    } else if |currentL| == 1 {
      currentL := [];
    } else {
      var x := currentL[0];
      var y := currentL[1];
      if x > y {
        return false;
      }
      currentL := currentL[1..];
    }
  }
}

```

Listing 3.30: `isSortedBI` Method

Proving the equivalence of imperative methods cannot be achieved directly through lemmas in Dafny; only functions can be called within lemmas as, by definition, Dafny functions are pure while Dafny methods are allowed to have side effects. Therefore, to prove equivalence, we must prove postconditions for both methods, and then state that these postconditions are equivalent. In turn, as in the above example, the user will often need to provide loop invariants and variants to help Dafny prove these postconditions.

These specifications can be very difficult to write correctly. Additionally, just proving that the methods' postconditions are equivalent is not sufficient to prove the equivalence of the methods themselves, as the postconditions may not fully capture the behaviour of the methods. *true*

3.7.1 Proving Equivalence of Imperative Methods using Product Programs

As described by Lahiri et al. [8], one solution to these difficulties is to generate product programs. A product program is a method that contains the bodies of the two methods being compared, in such a way that relationships between the two can be verified.

For example, we can use this process to prove the equivalence of the imperative versions of `isSortedR` and `isSortedB` (the function versions can be found in Listings 3.1 and 3.2 respectively). First, the imperative version of `isSortedR` is as follows, requiring a helper function `loopI`:

```

method loopI(p: int, l: seq<int>) returns (res: bool)
  ensures res <==> ((forall i :: 0 <= i < |l| - 1 ==> l[i] <= l[i+1]) &&
    (|l| > 0 ==> p <= l[0]))
{
  var currentP := p;
  var currentL := l;

  while true
    decreases |currentL|
    invariant currentL == l[|l| - |currentL| .. ]
    invariant currentP == if |l| == |currentL| then p
                          else l[|l| - |currentL| - 1]
    invariant |l| > |currentL| ==> p <= l[0]
    invariant forall i :: 0 <= i < |l| - |currentL| - 1 ==> l[i] <= l[i+1]
  {
    if |currentL| == 0 {
      return true;
    } else {
      var x := currentL[0];
      if currentP > x {
        return false;
      }
      currentP := x;
      currentL := currentL[1..];
    }
  }
}

method isSortedRI(l: seq<int>) returns (res: bool)
  ensures res <==> (forall i :: 0 <= i < |l| - 1 ==> l[i] <= l[i+1])
{
  if |l| == 0 {
    return true;
  } else {
    var x := l[0];
    var xs := l[1..];
    var result := loopI(x, xs);
    return result;
  }
}

```

Listing 3.31: isSortedRI Method with loopI Helper Method

These methods can be used to construct a product program as follows, using the definition of loopI given above:

```

method isSortedBI_RI_Equivalence(
  lb: seq<int>,
  lr: seq<int>
) returns (rres: bool, bres: bool)
  requires lb == lr
  ensures rres == bres
{
  // Defining output variables
  var rout: bool;
  var bout: bool;

  // Computing isSortedRI
  if |lr| == 0 {
    rout := true;
  } else {
    var x := lr[0];
    var xs := lr[1..];
    var result := loopI(x, xs);
    rout := result;
  }

  // Computing isSortedBI
  var currentL := lb;

  while true
    decreases |currentL|
    invariant currentL == lb[|lb| - |currentL| .. ]
    invariant forall i :: 0 <= i < |lb| - |currentL| ==>

```

```

                                (i < |lb| - 1 ==> lb[i] <= lb[i+1])
{
  if |currentL| == 0 {
    bout := true;
    break;
  } else if |currentL| == 1 {
    currentL := [];
  } else {
    var x := currentL[0];
    var y := currentL[1];
    if x > y {
      bout := false;
      break;
    }
    currentL := currentL[1..];
  }
}

// Returning the results
return rout, bout;
}

```

Listing 3.32: Product Program for isSortedBI and isSortedRI using loopI

interesting idea
(new to me)

The specification of this product program ensures that, since Dafny is able to verify this method, isSortedBI and isSortedRI must be equivalent. Therefore, this process removes the need for the user to provide postconditions for the individual methods, that capture their behaviour fully. In this case, a postcondition for the loopI helper function is still required, but the body of this method can also be copied into the product program, removing this requirement as well. However, the user must still provide the loop invariants and variants for the individual methods.

can you auto-gen these product programs (maybe without the invariants)?

3.7.2 Equivalence of Methods and Functions

A similar problem to that discussed above is proving the equivalence of a method and a function. In this case, the user does not need to write a postcondition for the method, since the function can be called directly within the specification. The user still needs to provide loop invariants and variants for the method, however, these do not need to be as complicated as those detailed above, as again, they can be defined in terms of the function. For example, the function isSortedB defined in Listing 3.2 can be proven to be equivalent with the following imperative method:

```

method isSortedBI(l: seq<int>) returns (res: bool)
  ensures res == isSortedB(l)
{
  var currentL := l;

  while true
    decreases |currentL|
    invariant isSortedB(currentL) == isSortedB(l)
  {
    if |currentL| == 0 {
      return true;
    } else if |currentL| == 1 {
      currentL := [];
    } else {
      var x := currentL[0];
      var y := currentL[1];
      if x > y {
        return false;
      }
      currentL := currentL[1..];
    }
  }
}

```

yes
much nicer/cleaner

Listing 3.33: isSortedBI Method with simplified specifications

Thus, the necessary specifications can be greatly simplified if they are defined in terms of the function the method is being compared to.

3.8 Equivalence of Higher-Order Functions

Dafny is able to prove the equivalence of some higher-order functions, but not all. For example, here are two equivalent functions `andThen1` and `andThen2`:

```
function andThen1<A, B, C>(f: A -> B, g: B -> C): A -> C { a => g(f(a)) }
function andThen2<A, B, C>(ff: A -> B, gg: B -> C): A -> C { aa => gg(ff(aa)) }
```

behavior?

Listing 3.34: `andThen1` and `andThen2` Functions

Dafny is able to prove their equivalence without any help:

```
lemma equivalenceAndThen<A, B, C>(f: A -> B, g: B -> C)
  ensures andThen1(f, g) == andThen2(f, g)
{}
```

Listing 3.35: Equivalence Lemma for `andThen1` and `andThen2`

Here are two more such functions `flip1` and `flip2`:

```
function flip1<A, B, C>(f: (A, B) -> C): (B, A) -> C {
  (b, a) => f(a, b)
}

function flip2<A, B, C>(f: (A, B) -> C): (B, A) -> C {
  (b, a) => var res := f(a, b); res
}
```

behavior?

Listing 3.36: `flip1` and `flip2` Functions

Dafny is able to prove that the outputted functions give the same output for the same inputs:

```
lemma equivalenceFlip<A, B, C>(f: (A, B) -> C)
  ensures forall b: B, a: A :: flip1(f)(b, a) == flip2(f)(b, a)
{}
```

Listing 3.37: Equivalence Lemma for `flip1` and `flip2` with specified arguments

However, it is unable to prove that the outputted functions themselves are equivalent:

```
// Dafny cannot prove this lemma
lemma equivalenceFlip<A, B, C>(f: (A, B) -> C)
  ensures flip1(f) == flip2(f)
{}
```

Listing 3.38: Invalid Equivalence Lemma for `flip1` and `flip2`

Therefore, Dafny is able to prove the equivalence of some higher-order functions. However, it struggles with others, despite being able to prove that the outputted functions give the same output for the same inputs.

any ideas where the limitations truly lie
(i.e. any pattern, to what it can/cannot prove)?

Chapter 4

Plans

4.1 Project Plan

The aim of this project is to build a tool on top of the Dafny program verifier that removes as many of the limitations identified in Chapter 3 as possible. Some of these limitations are out of the scope of this project. Preventing incorrect counterexamples (as described in Section 3.2) is one such limitation, as it results from problems in Boogie and the underlying Z3 solver. Similarly, the limitations related to imperative programs (as described in Section 3.7) are also out of scope, as Dafny functions provide enough complexity for this project by themselves.

I plan to build my tool in Scala. It will be called from the command line, taking as input multiple Dafny files, containing the functions to be compared and any help the user needs to provide. The tool will also require a JSON configuration file, specifying which functions must be proven to be equivalent and any other information the tool requires. The tool will attempt to create an equivalence proof and prove it using Dafny; Dafny's output will be used as the output of the tool.

The following table summarises the current development plan, broken down into milestones. Each milestone has an associated completion deadline and a description of what that milestone involves. Each milestone is described as either Necessary or Optional. If Necessary milestones take longer than I expected, I will use the time previously dedicated to Optional milestones to complete the Necessary ones.

If I have time remaining once all the project and evaluation milestones are completed, I plan to further extend our tool to handle the remaining limitations relating to Dafny functions identified in Chapter 3. This includes removing the necessity for the user to explicitly define induction schemes to prove lemmas (Section 3.5) and implementing the ability to prove user-provided auxiliary lemmas (Section 3.4).

Milestone	Description	Deadline	Necessity
1	Building the basic structure	09/02/2026	Necessary
2	Creating a CI script for Continuous Testing and Evaluation	16/02/2026	Necessary
3	Create Parser for Dafny functions	02/03/2026	Necessary
4	Copying Variants into the Main Lemma	09/03/2026	Optional
5	Generating Helper Equivalence Lemmas as Axioms	16/03/2026	Optional
6	Calling Helper Lemmas in the Main Lemma	23/03/2026	Optional
7	Calling Helper Lemmas in Equivalence Lemmas	26/03/2026	Optional
8	Proving Equivalence with one of a number of solutions	02/04/2026	Optional
9	Allowing the user to provide Auxiliary Lemmas as Axioms	06/04/2026	Optional
10	Proving equivalence of higher order functions	01/05/2026	Optional
11	Writing the Final Report	01/06/2026	Necessary

Table 4.1: Summary of the current development plan

Building the basic structure This milestone involves creating the skeleton of the tool. It should be able to take in Dafny files and a JSON configuration file. The Dafny files will contain the two functions being compared and an equivalence lemma without any user assistance. The tool will simply combine these files together, call the Dafny verifier on the resulting file, and return its output. Equivalence

lemmas with no user assistance are used here to provide an evaluation baseline for the evaluation of the remaining milestones.

Creating a CI script for Continuous Testing and Evaluation At each milestone, evaluation data must be generated and collected (see Section 4.2 for more details on this). This milestone involves creating a script to collect this data, and setting up a GitLab CI pipeline to automatically run this script on every merge. Once this is done, evaluation data should be collected for the previous milestone. Then, the equivalence lemmas should be replaced with lemmas that include all the user assistance that Dafny requires, followed by a recollection of the evaluation data, to serve as another baseline for the remaining milestones. ✓

Create Parser for Dafny functions Several of the future milestones will require an in depth understanding of the structure of the Dafny functions being compared and of the equivalence lemma provided by the user. Therefore, in this milestone, I will create parsers specifically for Dafny functions and lemmas. ✓

Copying Variants into the Main Lemma As described in Section 3.6, when one or both of the functions being compared require the user to explicitly provide variants, this must be copied into the main equivalence lemma. This milestone involves implementing this functionality. ✓

core | **Generating Helper Equivalence Lemmas as Axioms** This milestone will involve implementing the method of finding and generating equivalence lemmas for helper functions described by Milovančević and Kunčak [11] and summarised in Section 3.3. For now, these lemmas will be generated as axioms; they will be proven properly in a later milestone. ✓

core | **Calling Helper Lemmas in the Main Lemma** As described in Sections 3.3 and 3.6, helper equivalence and termination lemmas must be explicitly called in the main equivalence lemma, under the correct conditions and with the correct arguments. This milestone involves implementing this specifically for the main equivalence lemma. ✓

core | **Calling Helper Lemmas in Equivalence Lemmas** This milestone involves extending the functionality implemented in the previous milestone to also prove the generated helper equivalence lemmas. ✓

Proving Equivalence with one of a number of solutions In certain cases, including those described by Milovančević and Kunčak [11], the user may wish to prove the equivalence of a function with any one of a number of possible other functions. This milestone involves implementing this. ✓

Allowing the user to provide Auxiliary Lemmas as Axioms In certain cases, the user will need to provide auxiliary lemmas to help Dafny prove equivalence. These lemmas are specific to each individual scenario, and so automatic generation is out of scope for this project. Therefore, a reasonable alternative is to allow the user to provide auxiliary lemmas, which can then be called in the main equivalence lemma. These auxiliary lemmas will be added as axioms, as it would be difficult to prove them automatically. The burden of ensuring these lemmas are correct will be on the user. This milestone will involve implementing this. *are these not just added to the Dafny code input - seems trivial to support...*

Proving equivalence of higher order functions When proving the equivalence of functions that output lambda functions, these functions are equivalent if the lambda functions output the same values for the same inputs. This can be proven through a universal quantification over the domain of the lambda functions. This milestone involves implementing this functionality. ✓

Writing the Final Report Throughout development, I will add to the Final Report as I go along; this milestone involves completing the technical chapters in it. ✓

4.2 Evaluation Plan

In a similar manner to Section 4.1, here is a table summarising the current evaluation plan, broken down into milestones.

Milestone	Description	Deadline	Necessity
1	Creating Initial Evaluation Testsuite	Completed	Necessary
2	Setup of Evaluation Data Gathering	16/02/2026	Necessary
3	Testsuite Development	08/05/2026	Optional
4	Benchmark Comparison	11/05/2026	Necessary
5	Writing the Final Report	05/06/2026	Necessary

Table 4.2: Summary of the current evaluation plan

this is not part of eval plan

Creating Initial Evaluation Testsuite Before evaluation can begin, we need a testsuite containing pairs of functions which our tool will attempt to prove equivalent. As described in Chapter 3, I have already created an initial testsuite using the equivalence tests from the repository for the Stainless Scala verification tool[14]. This testsuite will be extended in a later milestone. *nice to have a precise testset*

Setup of Evaluation Data Gathering As described in Section 4.1, once each milestone is completed, a CI script will be used to automatically collect evaluation data. The first two milestones will provide baseline data on Dafny without any user assistance, and on Dafny with all the assistance it requires respectively. Data collected from all subsequent milestones will be compared to this baseline data. The evaluation data will consist of two categories. First, we must gather data to represent how difficult each example is for Dafny to verify. This category will include the metrics of the number of lines of code in each function being compared, the number of helper functions called and the time taken for Dafny to verify the example. Second, we must gather data to represent how much user assistance Dafny requires. This will include the metrics of the number of lines of code in the main equivalence lemma and the numbers of helper equivalence and auxiliary lemmas the user needs to provide. The aim of this project is for the metrics in the second category to decrease as much as possible. The first category metrics will ensure that we are interpreting these decreases in the correct context. These metrics are too shallow to provide a complete picture of the complexity of the examples and the level of user assistance required, however, they will provide a reasonable approximation.

Testsuite Development A second set of program equivalence tests have been gathered by Badihi, Li, and Rubin [1]. This milestone involves converting these tests from C and Java into Dafny, and appending them into my evaluation testsuite. *good idea*

Benchmark Comparison In this milestone, I will compare the results of our tool on our evaluation testsuite with the results from the original Dafny verifier, to demonstrate the improvements made. I will also compare our tool with Stainless on the subset of our testsuite that originated from their equivalence tests, to demonstrate how our tool compares with other state-of-the-art program verifiers.

Writing the Final Report Throughout development, I will add to the Final Report as I go along; this milestone involves completing the evaluation chapters in it.

not eval

how measure "level of user assistance"

different kind of user annotations have different difficulty levels to the user

+ some system profiling

Chapter 5

Ethical, Legal, Professional and Societal Issues

This project raises very few ethical, legal, professional and societal issues. The only potential issue is that our tool must obey the licenses of the any third-party libraries or tools it uses. In particular, our tool must obey Dafny's MIT license, since Dafny will be used to verify the final equivalence lemmas. It must also obey Stainless's Apache 2.0 license[14] and EqBench's MIT license[1], since these repositories ✓ will be used to form our evaluation testsuite.

what about environmental impact?

how sustainable is your tool - what are the power consumption overheads?

soundness? ← responsibility to not
say program is correct if
they are not

Bibliography

- [1] Sahar Badihi, Yi Li, and Julia Rubin. “EqBench: A Dataset of Equivalent and Non-equivalent Program Pairs”. In: *2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*. 2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR). Madrid, Spain: IEEE, May 2021, pp. 610–614. ISBN: 978-1-7281-8710-5. DOI: [10.1109/MSR52588.2021.00084](https://doi.org/10.1109/MSR52588.2021.00084). URL: <https://ieeexplore.ieee.org/document/9463140/> (visited on 01/16/2026).
- [2] Mike Barnett et al. “Boogie: A Modular Reusable Verifier for Object-Oriented Programs”. In: *Formal Methods for Components and Objects*. Ed. by Frank S. de Boer et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2006, pp. 364–387. ISBN: 978-3-540-36750-5.
- [3] Cristian Cadar, Daniel Dunbar, and Dawson Engler. “KLEE: Unassisted and Automatic Generation of High-Coverage Tests for Complex Systems Programs”. In: ().
- [4] Leonardo de Moura and Nikolaj Bjørner. “Z3: An Efficient SMT Solver”. In: *Tools and Algorithms for the Construction and Analysis of Systems*. Ed. by C. R. Ramakrishnan and Jakob Rehof. Berlin, Heidelberg: Springer Berlin Heidelberg, 2008, pp. 337–340. ISBN: 978-3-540-78800-3.
- [5] *Epfl-Lara/Inox*. EPFL-LARA, Jan. 14, 2026. URL: <https://github.com/epfl-lara/inox> (visited on 01/20/2026).
- [6] *Introduction — Stainless 0.9.1 Documentation*. URL: <https://epfl-lara.github.io/stainless/intro.html> (visited on 01/20/2026).
- [7] Shuvendu K Lahiri et al. “Automated Differential Program Verification for Approximate Computing”. In: ().
- [8] Shuvendu K. Lahiri et al. “Differential Assertion Checking”. In: *Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering. ESEC/FSE 2013*. New York, NY, USA: Association for Computing Machinery, Aug. 18, 2013, pp. 345–355. ISBN: 978-1-4503-2237-9. DOI: [10.1145/2491411.2491452](https://doi.org/10.1145/2491411.2491452). URL: <https://doi.org/10.1145/2491411.2491452> (visited on 12/17/2025).
- [9] Shuvendu K. Lahiri et al. “SYMDIFF: A Language-Agnostic Semantic Diff Tool for Imperative Programs”. In: *Computer Aided Verification*. Ed. by P. Madhusudan and Sanjit A. Seshia. Red. by David Hutchison et al. Vol. 7358. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 712–717. ISBN: 978-3-642-31423-0 978-3-642-31424-7. DOI: [10.1007/978-3-642-31424-7_54](https://doi.org/10.1007/978-3-642-31424-7_54). URL: http://link.springer.com/10.1007/978-3-642-31424-7_54 (visited on 11/25/2025).
- [10] K. Rustan M. Leino. “Dafny: An Automatic Program Verifier for Functional Correctness”. In: *Logic for Programming, Artificial Intelligence, and Reasoning*. Ed. by Edmund M. Clarke and Andrei Voronkov. Vol. 6355. Berlin, Heidelberg: Springer Berlin Heidelberg, 2010, pp. 348–370. ISBN: 978-3-642-17510-7 978-3-642-17511-4. DOI: [10.1007/978-3-642-17511-4_20](https://doi.org/10.1007/978-3-642-17511-4_20). URL: http://link.springer.com/10.1007/978-3-642-17511-4_20 (visited on 11/20/2025).
- [11] Dragana Milovančević and Viktor Kunčák. “Proving and Disproving Equivalence of Functional Programming Assignments”. In: *Proceedings of the ACM on Programming Languages* 7 (PLDI June 6, 2023), pp. 928–951. ISSN: 2475-1421. DOI: [10.1145/3591258](https://doi.org/10.1145/3591258). URL: <https://dl.acm.org/doi/10.1145/3591258> (visited on 11/20/2025).
- [12] Can Özkan. *KLEE: Smarter Test Generation for Reliable Code*. Medium. Aug. 15, 2025. URL: <https://can-ozkan.medium.com/klee-smarter-test-generation-for-reliable-code-dfbd0f89fb44> (visited on 01/20/2026).

- [13] Luciano Putruele et al. “MaskD: A Tool for Measuring Masking Fault-Tolerance”. In: *Tools and Algorithms for the Construction and Analysis of Systems*. Ed. by Dana Fisman and Grigore Rosu. Vol. 13243. Cham: Springer International Publishing, 2022, pp. 396–403. ISBN: 978-3-030-99523-2 978-3-030-99524-9. DOI: [10.1007/978-3-030-99524-9_22](https://doi.org/10.1007/978-3-030-99524-9_22). URL: https://link.springer.com/10.1007/978-3-030-99524-9_22 (visited on 11/24/2025).
- [14] *Stainless/Frontends/Benchmarks/Equivalence at Main · Epfl-Lara/Stainless*. GitHub. URL: <https://github.com/epfl-lara/stainless/tree/main/frontends/benchmarks/equivalence> (visited on 01/10/2026).

nice set of paper references